

|                                                                   |                          |
|-------------------------------------------------------------------|--------------------------|
| <b>Name of Student: Aryyan Paygude</b>                            | <b>PRN:25070122291</b>   |
| <b>Semester: III</b>                                              | <b>Year : AY 2026-27</b> |
| <b>Subject: Programming with Java</b>                             |                          |
| <b>ASSIGNMENT No : 1</b>                                          |                          |
| <b>TITLE : Java Environment Setup and Basic Program Execution</b> |                          |

### **Problem Statement**

Develop an application based on Java environment setup, JDK/JRE/JVM, variables, methods and constructors. and demonstrate the implementation of the concepts through suitable examples.

### **Learning Objectives**

Understand the theoretical concepts, implement them in Java programs, analyse outputs, and apply the concepts to solve practical programming problems.

### **Programme**

```
public class StudentDemo {
    // instance variables
    String name;
    int rollNo;
    double marks;

    // parameterized constructor
    StudentDemo(String name, int rollNo, double marks) {
        this.name = name;
        this.rollNo = rollNo;
        this.marks = marks;
    }

    // default constructor (constructor overloading)
    StudentDemo() {
        this.name = "Not Assigned";
        this.rollNo = 0;
        this.marks = 0.0;
    }

    // method to display student details
    void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
    }
}
```

```

        System.out.println("Marks: " + marks);
    }

    // method that returns a value
    double calculatePercentage() {
        return (marks / 500) * 100;
    }

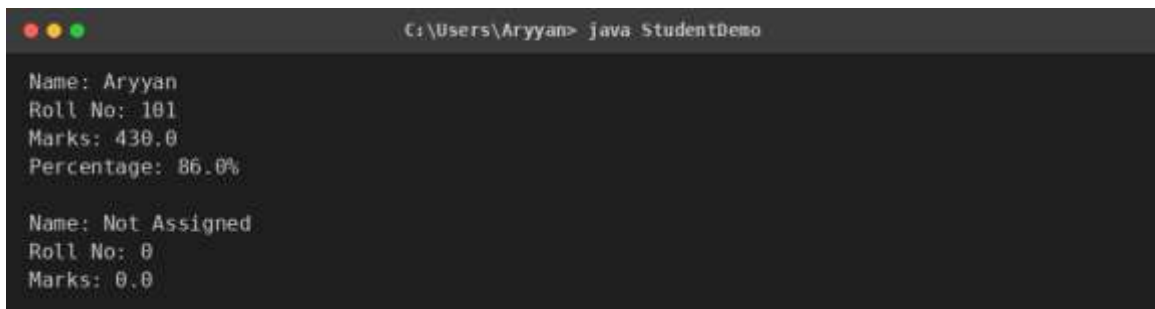
    public static void main(String[] args) {
        // object created using parameterized constructor
        StudentDemo s1 = new StudentDemo("Aryyan", 101, 430.0);
        s1.displayDetails();
        System.out.println("Percentage: " + s1.calculatePercentage() + "%");

        System.out.println();

        // object created using default constructor
        StudentDemo s2 = new StudentDemo();
        s2.displayDetails();
    }
}

```

## Output



```

C:\Users\Aryyan> java StudentDemo
Name: Aryyan
Roll No: 101
Marks: 430.0
Percentage: 86.0%

Name: Not Assigned
Roll No: 0
Marks: 0.0

```

## Conclusion

Thus, the Java environment (JDK/JRE/JVM) was successfully set up and a Java program was developed and executed to demonstrate variables, methods, and constructors. The program showed how instance variables store an object's data, how methods define reusable behaviour, and how constructor overloading (default and parameterized) allows flexible object initialization. This helped in understanding how Java's platform-independent execution model (source code → bytecode → JVM) works alongside core object-oriented building blocks.

## Questions

1. Explain the main concept used in Java environment setup, variables and methods.

*Ans: Java environment setup involves installing the JDK (Java Development Kit), which bundles the JRE (Java Runtime Environment) and the JVM (Java Virtual Machine). The JDK provides the compiler (javac) used to convert .java source files into platform-independent bytecode (.class files), while the JVM executes that bytecode on any operating system. Variables are named memory locations that hold data of a specific type (e.g., int, double, String), and methods are named blocks of code that perform a specific task and can be called (reused) whenever needed, optionally taking parameters and returning a value.*

2. What are the advantages of using Java variables and methods?

*Ans: Variables allow data to be stored, labelled, and reused throughout a program instead of using raw hard-coded values, and Java's static typing catches type-related errors at compile time itself. Methods promote code reusability (write once, call many times), improve readability by breaking a program into logical, named chunks, make debugging easier since each method can be tested independently, and support modular design, which is essential as programs grow larger.*

3. How can this implementation be improved?

*Ans: This implementation can be improved by making the instance variables private and adding getter/setter methods (proper encapsulation) instead of accessing them directly, adding input validation (e.g., rejecting negative marks or roll numbers), using exception handling to manage invalid data gracefully, and adding a copy constructor to allow duplicating existing student records.*

4. What are the real-world applications?

*Ans: This concept is the foundation of real-world Java applications such as student/employee management systems, banking applications (account objects with balance, deposit/withdraw methods), e-commerce platforms (product objects with price, stock, and constructors for initialization), and any enterprise software where data needs to be modelled as objects with defined behaviour.*